SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	*Hitesh Roy*	**Reg. No.:**	192372036
Section / Batch:	CSE(AI)-2023	**Date:**	20/7/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – CONCEPT MAPPING

Code of Conduct

I Hitesh Roy (192372036) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Hitesh Roy*

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

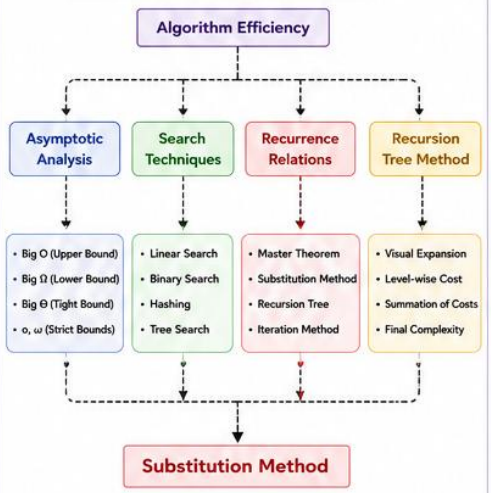
Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SUBSTITUTION METHOD FOR RECURRENCE RELATIONS

A mathematical technique to solve recurrences by assuming a bound and proving it using induction.

ALGORITHM EFFICIENCY MAP



SUBSTITUTION METHOD – SYSTEMATIC FLOW

1 Understand Recurrence

Write the recurrence in standard form $T(n) = aT(n/b) + f(n)$ or other form.

2 Assume Solution

Make an educated guess for the upper (or lower) bound. Typically, $T(n) \le c \cdot g(n)$ or $T(n) \ge c \cdot g(n)$

3 Expand Recurrence

Substitute the assumed bound into recurrence and simplify using algebra. Derive an inequality of the form $T(n) \le \dots$

4 Base Case

Verify the assumed bound for the smallest input(s) $(n = n_0)$.

5 Induction Hypothesis

Assume the solution is true for all values less than n. That is, $T(k) \le c \cdot g(k)$ for all $n_0 \le k < n$.

6 Induction Step (Proof)

Prove that the assumed bound holds for n, i.e., show that $T(n) \le c \cdot g(n)$ (or $T(n) \ge c \cdot g(n)$). Choose c large enough if required.

7 Verified Complexity

If base case and induction step are true, by mathematical induction, the bound holds for all $n \ge n_0$. Thus, time complexity is verified.

EXTENDED MAP WITH INDUCTION COMPONENTS

Step	Component	Purpose
1	Understand Recurrence	Identify the form and structure of the recurrence.
2	Assume Solution	Propose a bound (e.g., $O(g(n))$ or $\Omega(g(n))$).
3	Expand Recurrence	Use the assumption to bound $T(n)$ in terms of $g(n)$.
4	Base Case	Show the bound is true for small n $(n = n_0)$.
5	Induction Hypothesis	Assume bound holds for all $k < n$.
6	Induction Step	Prove the bound holds for n using the hypothesis.
7	Verified Complexity	Conclude the bound is true for all $n \ge n_0$.

WORKED EXAMPLE USING SUBSTITUTION METHOD

Example Recurrence:

$T(n) = 2T(n/2) + n \quad \text{for } n \ge 2, \quad T(1) = 1$

1. **Assume Solution:** Assume $T(n) \le cn \log n$ for some constant $c > 0$.
2. **Base Case:** For $n = 1$,
   $T(1) = 1 \le c \cdot 1 \cdot \log 1 = 0$ (true for $c \ge 1$)
3. **Induction Hypothesis:** Assume $T(k) \le ck \log k$ for all $1 \le k < n$.
4. **Induction Step:**
   $$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\le 2\left[c\left(\frac{n}{2}\right)\log\left(\frac{n}{2}\right)\right] + n \quad \text{(by IH)} \\ &= cn(\log n - 1) + n \\ &= cn \log n - cn + n \\ &\le cn \log n \qquad \text{if } c \ge 1 \end{aligned}$$
   Hence, $T(n) \le cn \log n$.
5. **Conclusion:** Since base case is true and induction step holds, by induction, $T(n) = O(n \log n)$. Also, using Master Theorem (Case 2), $T(n) = \Theta(n \log n)$.

Key Points

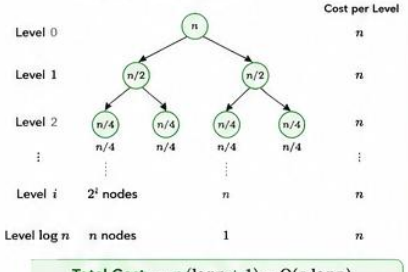
- Choose the correct form of $g(n)$.
- Find an appropriate constant c.
- Simplify carefully after substitution.
- Prove inequality for n, not just simplify.

Common Guesses ($g(n)$)

Recurrence Pattern	Typical Guess
$T(n) = aT(n/b) + O(1)$	$O(n^{\log_b a})$
$T(n) = aT(\frac{n}{b}) + O(n^k)$ $k < \log_b a$	$O(n^{\log_b a})$
$T(n) = aT(\frac{n}{b}) + \Theta(n^{\log_b a}\log^k n)$	$O(n^{\log_b a}\log^{k+1} n)$
$T(n) = aT(\frac{n}{b}) + \Omega(n^k)$ $k > \log_b a$	$O(n^k)$

RECURSION TREE METHOD (VISUAL ALTERNATIVE)

For $T(n) = 2T(n/2) + n$



Total Cost $= n(\log n + 1) = \Theta(n \log n)$.

WHEN TO USE WHICH METHOD?

Master Theorem
- Recurrence in form $T(n) = aT(n/b) + f(n)$
- $f(n)$ matches one of the three cases
- Easiest and fastest

Substitution Method
- Master Theorem not applicable
- Need tighter bound (upper/lower)
- Recurrence not in standard form

Recursion Tree Method
- Need intuition or visualization
- Non-uniform work at different levels
- Good for teaching and verification

WHY PROOF (INDUCTION) IS NECESSARY?

Guarantees Correctness
Induction ensures the assumed bound is valid for all input sizes, not just a few.

Avoids Wrong Assumptions
A wrong guess may hold for small n but fail for larger n. Proof prevents this.

Mathematical Rigor
Provides a formal and general justification acceptable in theoretical analysis.

Enables Reliable Analysis
Critical in algorithm design, optimization, and performance prediction.

QUICK SUMMARY

Guess Bound → Expand & Simplify → Base Case ($n = n_0$) → Induction Hypothesis ($k < n$) → Induction Step (Show for n) → By Induction True for all $n \ge n_0$ → Complexity Verified

The substitution method converts "guesswork" into "proof" using mathematical induction.

TIPS FOR SUCCESS

- Practice solving many recurrences.
- Be careful with algebra and inequalities.
- Choose c large enough when needed.
- Always verify base case.
- Check your result with intuition or other methods.